



Yet Another Stupid Simple AI agent.

Correct first, then ruthlessly token-efficient.

19/19

Matched real-task judge
accuracy

4,826

Total Fireworks tokens

2

Model calls for all tasks

0

Reasoning tokens and fallbacks

Go 1.26

MiniMax-M3

Python 3 tool execution

Granite ModernBERT ONNX

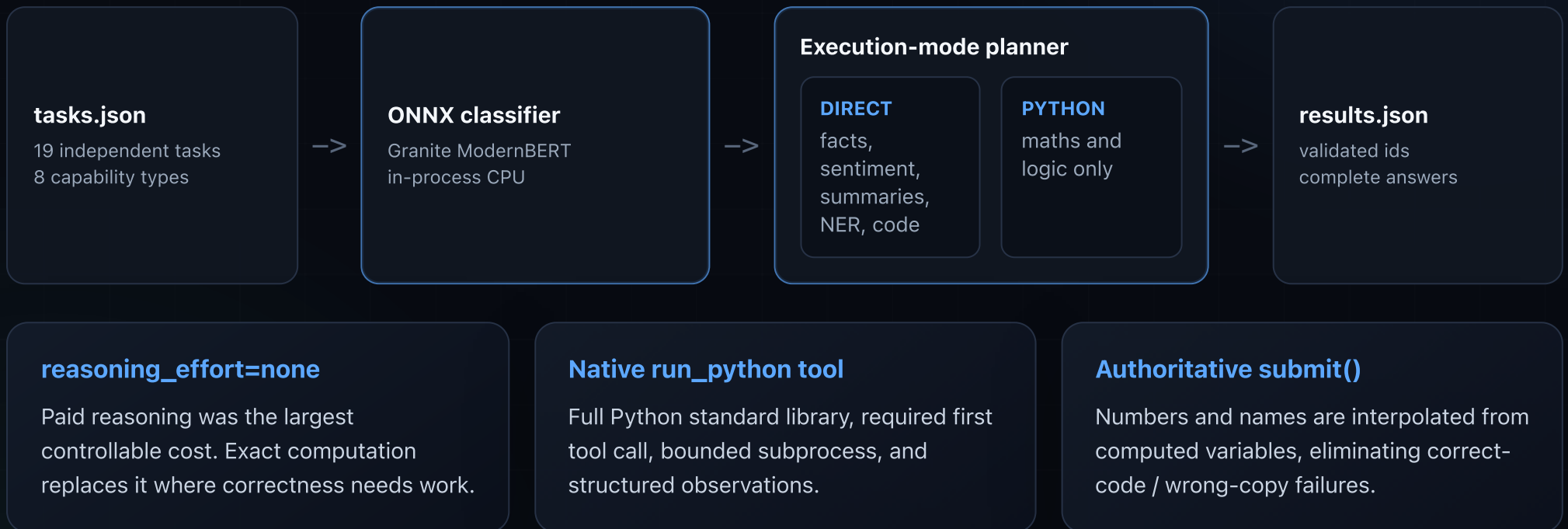
linux/amd64

Git LFS

Track 1 ranks the fewest Fireworks tokens among submissions that clear the accuracy gate.

Two calls. One deterministic escape hatch.

The classifier decides execution mode locally; it does not spend Fireworks tokens.



Semantic routing that survives messy prompts.

Model and runtime

- + IBM Granite Embedding R2 backbone, ~47M parameters
- + 8-way multi-label classification head
- + ModernBERT, 1,024 tokens, head+tail truncation
- + int8 ONNX, ~48 MB, in-process Go inference
- + Native amd64 verified; large models stored with Git LFS

misspelling augmentation

semantic categories

AMD ROCm training

Held-out hard benchmark

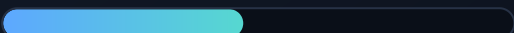
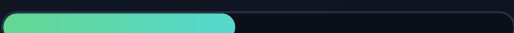
METRIC	V4
micro-F1	0.977
macro-F1	0.967
subset accuracy	0.930
previous MiniLM v1 micro-F1	0.889

Labels cost roughly 10 tokens per task but previously measured about 12% lower total Fireworks use by focusing answer recipes and execution mode.

- factual
- maths
- sentiment
- summarisation
- NER
- debugging
- logic
- code generation

The path from unstable to 19/19.

Measured run history

broken base		10,659
lean effort		6,046
prompt trim		5,773
two batches		4,999
final		4,826

What actually moved the score

- + Disabled reasoning tokens in every category
- + Moved maths and logic into one real-Python batch
- + Forced computed-variable interpolation in submit()
- + Merged all other categories into one direct batch
- + Kept terse, category-specific answer recipes
- + Matched the organiser-style reference judge locally

```
prompt + output + reasoning = ranked tokens
3,013 + 1,813 + 0           = 4,826
```

The leaderboard target was ~4,300. The current cloud-only result is 4,826 at 19/19; further reduction needs a reliable zero-token local offload, not weaker answers.

Useful at 500+ tokens - gated by accuracy.

Fireworks prompt-token measurements

SOURCE	TEXT	IMAGE	SAVED
500 chars	262	216	18%
2,000 chars	645	312	52%
5,000 chars	1,390	528	62%
20,000 chars	5,141	1,632	68%
40,000 chars	10,144	3,096	69%

19,250-character probe: 4,312 -> 1,421 prompt tokens, 2/2 answers preserved.

Production auto mode

- + Keep ids, questions, constraints, code, maths, and logic as text
- + Render only quoted sources $\geq 2,000$ characters (~500 tokens)
- + Label every image immediately before its image_url wire part
- + Fall back to text on threshold, render, or mode mismatch
- + Keep aggressive tasks/dense/full modes for research

Rejected experiment: two ~450-character images saved only 13 prompt tokens and scored 18/19. Auto now stays text on the real set, preserving 19/19.

A reproducible submission, not a notebook demo.

Deterministic CI

- + Go 1.26 matches go.mod and Docker
- + Native amd64 ONNX integration test
- + Network experiments isolated from required CI
- + content:null handled without panics

Portable image

- + linux/amd64 multi-stage Docker build
- + Static libtokenizers + bundled ONNX Runtime
- + Python 3 runtime for deterministic tools
- + GHCR latest, main, and immutable SHA tags

Telemetry loop

- + Per-call prompt/output/reasoning tokens
- + Batch plans, classifications, and tool traces
- + KV-backed Cloudflare dashboard
- + Token history exposes regressions immediately

PASS

Matched accuracy gate

amd64

Judge-native target

manual

Live text2img workflow

ready

Commit, image, and docs